

Źródła, struktury i formaty danych

Paweł Gliwny

Tematy na dziś

- Struktura rekordów i schematy danych
- Tekstowe i binarne formaty zapisu danych
- Dane tabelaryczne, hierarchiczne i kolumnowe
- Wczytywanie danych z plików, API i baz danych
- Kodowanie znaków

Format a źródło

Format — jak dane są zapisane (CSV, JSON, Parquet, ...)

Źródło — skąd je bierzemy (plik lokalny, API, baza danych)

Te dwa wymiary są niezależne.

Ten sam zbiór danych może być zapisany w różnych formatach, a ten sam format może pochodzić z różnych źródeł.

Wybór formatu i źródła to jedna z pierwszych decyzji w projekcie analitycznym — i jedna z tych, które najbardziej wpływają na późniejszą wydajność i wygodę pracy.

Rekord i schemat

Rekord — pojedyncza obserwacja. Wiersz w tabeli, obiekt w JSON, dokument w bazie.

Schemat — opis struktury rekordu: jakie ma pola, jakich typów, jakie ograniczenia.

Przykład schematu:

pomiar:

station_id: tekst, wymagane

date: data, wymagane

pm25: liczba zmiennoprzecinkowa, ≥ 0

pm10: liczba zmiennoprzecinkowa, ≥ 0

Schema-on-write vs schema-on-read

Schema-on-write — schemat narzucony przy zapisie. Tak działają bazy SQL i Parquet. Dane „brudne” nie wejdą do bazy.

- Plus: spójność, błędy ujawniają się od razu
- Minus: sztywność, trzeba zaplanować schemat z góry

Schema-on-read — schemat narzucany dopiero przy odczycie. Tak działają CSV, JSON, data lakes.

- Plus: elastyczność, łatwo wrzucić nowe dane
- Minus: błędy ujawniają się późno, każdy konsument musi sam parsować

Współczesna analityka często łączy oba podejścia: **dane surowe lądują jako pliki (schema-on-read)**, a po **oczyszczeniu** trafiają do Parquet lub bazy **(schema-on-write)**.

Płaski vs zagnieżdżony rekord

Te same dane można zapisać na różne sposoby:

Płaski (CSV, tabela SQL):

station_id	date	pm25	pm10
PL001	2024-05-01	12.3	18.5
PL001	2024-05-02	15.1	22.0

Zagnieżdżony (JSON):

```
{  
  "station": {"id": "PL001", "city": "Łódź"},  
  "measurements": [  
    {"date": "2024-05-01", "pm25": 12.3, "pm10": 18.5},  
    {"date": "2024-05-02", "pm25": 15.1, "pm10": 22.0}  
  ]  
}
```

Wybór zależy od natury danych. Tabelaryczne nadają się do analiz statystycznych, hierarchiczne lepiej oddają strukturę obiektów ze świata rzeczywistego (typowe dla API).

Formaty tekstowe — przegląd

- **CSV / TSV** — proste tabele, wartości rozdzielone separatorem
- **JSON** — dane hierarchiczne, standard API
- **XML** — hierarchiczne, z metadanymi, dziś rzadziej używane
- **TXT, LOG** — niestukturalny tekst, logi

Są to pliki tekstowe, które można otworzyć w dowolnym edytorze.

Czytelne dla człowieka, ale mniej wydajne dla maszyny.

Funkcje wczytujące dane do DataFrame

Pandas ma całą rodzinę funkcji do wczytywania danych tabelarycznych:

- `read_csv`,
- `read_json`,
- `read_excel`,
- `read_parquet`,
- `read_sql` i wiele innych.

Wszystkie zwracają obiekt DataFrame.

Funkcje wczytujące dane do DataFrame (2)

Funkcje te przyjmują dziesiątki argumentów, które można pogrupować w pięć kategorii:

- **Indeksowanie** — wybór kolumn i nazw nagłówków
- **Typy i konwersja danych** — jak interpretować wartości
- **Parsowanie daty i czasu** — zamiana tekstu na daty
- **Iteracja** — wczytywanie porcjami
- **Radzenie sobie z „brudnymi” danymi** — pomijanie, czyszczenie

Znajomość tych kategorii pozwala szybciej znaleźć właściwy parametr w dokumentacji zamiast pisać kod, który „obchodzi” problem ręcznie.

Kategorie parametrów

Indeksowanie

- Wybór kolumn, które mają trafić do DataFrame (`usecols`)
- Ustawienie jednej lub kilku kolumn jako indeks (`index_col`)
- Wczytanie nazw kolumn z pliku, podanie własnych lub rezygnacja (`header, names`)

Typy i konwersja danych

- Wymuszenie typów kolumn (`dtype`)
- Własne funkcje konwertujące (`converters`)
- Oznaczenia brakujących wartości (`na_values`)

Parsowanie daty i czasu

- Zamiana tekstu na datetime (`parse_dates`)
- Łączenie kilku kolumn w jedną datę (np. osobne rok, miesiąc, dzień)
- Format daty (`date_format`)

Iteracja

- Wczytywanie pliku w porcjach zamiast całości (`chunksize`)
- Tryb iteracyjny dla plików nie mieszczących się w RAM (`iterator=True`)

CSV — Comma-Separated Values

Najprostszy format tabelaryczny. Każdy wiersz to jeden rekord, wartości rozdzielone przecinkiem (lub innym separatorem).

```
id,imie,miasto,wiek
```

```
1,Anna,Łódź,34
```

```
2,Bartek,Warszawa,28
```

```
3,Celina,Kraków,45
```

Zalety: prostota, czytelność, uniwersalność, obsługiwany wszędzie.

Wady: brak typów (wszystko jest tekstem), brak schematu, duży rozmiar, problemy z separatorami i cudzysłowami.

CSV — typowe pułapki

- **Separator** — przecinek, średnik, tabulator? Polski Excel domyślnie zapisuje średnikami.
- **Cudzysłowy** — co jeśli wartość sama zawiera przecinek lub cudzysłów?
- **Znak końca linii** — `\n` (Unix) czy `\r\n` (Windows)?
- **Nagłówek** — czy pierwszy wiersz to nazwy kolumn?
- **Kodowanie** — UTF-8, CP1250, ISO-8859-2?
- **Brakujące wartości** — pusty string, NA, NULL, -?

Mimo tych pułapek CSV pozostaje najczęściej używanym formatem wymiany danych.

Radzenie sobie z „brudnymi” danymi

- Pomijanie wierszy na początku (`skiprows`) i końcu (`skipfooter`)
- Pomijanie komentarzy (`comment="#"`)
- Separator tysięcy (`thousands=" "` — w polskich plikach często spacja)
- Separator dziesiętny (`decimal=","`)
- Kodowanie (`encoding`)

Szczegóły w dokumentacji: `pd.read_csv`

Kodowanie znaków — po co?

Komputer przechowuje tekst jako liczby. **Kodowanie** to umowa, która mówi, jaka liczba (lub sekwencja bajtów) odpowiada jakiemu znakowi.

Bez znajomości kodowania ten sam plik tekstowy może być nieczytelny — zamiast Łódź zobaczymy Å, ÃłdÃş. To zjawisko nazywa się **mojibake**.

Kodowanie znaków to jeden z najczęstszych powodów błędów przy wczytywaniu danych — szczególnie w polskim kontekście.

Krótką historia kodowań

- **ASCII** (1963) — 7 bitów, 128 znaków. Wystarcza dla angielskiego, brak polskich liter.
- **Strony kodowe 8-bitowe** — rozszerzenia ASCII o znaki narodowe. Dla polskiego: ISO-8859-2, Windows-1250, CP852, Mazovia. Każda inaczej koduje ą, ę, ł.
- **Unicode** — wspólny katalog wszystkich znaków świata (ponad 150 tys.). Każdy znak ma swój numer (code point), np. ł to U+0141.
- **UTF-8** — najpopularniejsze kodowanie Unicode. Zmiennej długości (1–4 bajty na znak), wstecznie zgodne z ASCII. Współczesny standard de facto.
- **UTF-16** — używane wewnętrznie m.in. w Windows i Javie.

Mojibake — kiedy kodowania się mylą

Tekst Łódź zapisany w różnych kodowaniach:

Kodowanie	Bajty (hex)
UTF-8	C5 81 C3 B3 64 C5 BA
Windows-1250	A3 F3 64 9F
ISO-8859-2	A3 F3 64 BF

Jeśli plik zapisany w Windows-1250 odczytamy jako UTF-8 — dostaniemy błąd lub krzaczki. I odwrotnie.

Wniosek: zawsze trzeba wiedzieć (lub odgadnąć), w jakim kodowaniu jest plik.

API — dane na żądanie

API (Application Programming Interface) — interfejs, przez który aplikacja udostępnia swoje dane lub funkcje innym programom.

W kontekście danych najczęściej spotykamy **REST API**: klient wysyła zapytanie HTTP (zwykle GET), serwer odpowiada danymi w formacie JSON.

Cechy API jako źródła danych:

- Dane są **aktualne** — pobierane na żądanie
- Często wymagają **autoryzacji** (klucz API, token)
- Mają **limity zapytań** (rate limiting, np. 60/min)
- Wyniki bywają **stronicowane** (paginacja)
- Format odpowiedzi: zwykle **JSON**, czasem XML lub CSV

Jak wygląda zapytanie HTTP

Najprostsze zapytanie do API składa się z trzech elementów:

- **URL** — adres zasobu, np. `https://api.nbp.pl/api/exchangerates/rates/A/EUR/`
- **Metoda HTTP** — zwykle GET (pobieranie), rzadziej POST, PUT, DELETE
- **Nagłówki** — opcjonalnie klucz API, format odpowiedzi, itp.

W Pythonie używamy biblioteki requests:

```
import requests

url = "https://api.nbp.pl/api/exchangerates/rates/A/EUR/last/10/"
response = requests.get(url)

print(response.status_code) # 200 = OK, 404 = nie znaleziono
print(response.headers["Content-Type"]) # application/json
data = response.json() # parsowanie JSON do dict/list
```

Przykład — API NBP

Narodowy Bank Polski udostępnia kursy walut przez otwarte API bez klucza — idealne do nauki.

Endpoint zwraca JSON z listą notowań. Funkcja `pd.json_normalize` spłaszcza strukturę do DataFrame.

Za każdym wywołaniem dostajemy **aktualne** dane — to przewaga API nad plikiem.

```
import requests
import pandas as pd

url = ("https://api.nbp.pl/api/"
      "exchangerates/rates/A/EUR/"
      "last/10/")

r = requests.get(url)
data = r.json()

df = pd.DataFrame(data["rates"])
df["effectiveDate"] = pd.to_datetime(
    df["effectiveDate"]
)
df = df.set_index("effectiveDate")
print(df.head())
```

Praktyczne wyzwania API

- **Autoryzacja** — większość API wymaga klucza w nagłówku lub parametrze. Klucze trzymaj w zmiennych środowiskowych (`os.environ`), **nigdy** w kodzie na GitHubie.
- **Rate limiting** — np. 60 zapytań/minutę. Przekroczenie → błąd 429 Too Many Requests. Rozwiązanie: `time.sleep()`, biblioteka `tenacity` z automatycznym ponawianiem.
- **Paginacja** — API nie zwraca wszystkiego naraz, tylko stronami po np. 100 rekordów. Trzeba pętlą pobierać kolejne strony aż do wyczerpania.
- **Zmiany API** — wersjonowanie (`/v1/`, `/v2/`), zmiany schematu, deprecjacja endpointów. Zawsze czytaj dokumentację i sprawdzaj wersję.

- **Etyka i regulamin** — nie każde API wolno swobodnie używać. RODO, warunki użytkowania, robots.txt.

Polskie otwarte API warte zapamiętania: NBP (kursy), GIOS (jakość powietrza), GUS BDL (statystyki), dane.gov.pl

JSON — JavaScript Object Notation

Format tekstowy dla danych hierarchicznych. Standard w komunikacji z API.

```
{  
  "station": {"id": "PL001", "city": "Łódź"},  
  "measurements": [  
    {"date": "2024-05-01", "pm25": 12.3},  
    {"date": "2024-05-02", "pm25": 15.1}  
  ]  
}
```

JSON — zalety i wady

Zalety: obsługuje zagnieżdżenia, ma typy podstawowe (liczba, string, bool, null), czytelny.

Wady: trudniejszy do wczytania jako tabela — wymaga „spłaszczania” (`pd.json_normalize`).

Po co spłaszczanie? JSON z API ma strukturę drzewa — obiekty w obiektach, listy obiektów. Pandas potrzebuje tabeli: wiersze i kolumny.

Funkcja `pd.json_normalize` zamienia zagnieżdżoną strukturę na płaską tabelę, tworząc nazwy kolumn z kropkami (np. `station.city`).

```
import pandas as pd

dane = [
    {"station": {"id": "PL001", "city": "Łódź"},
     "pm25": 12.3, "date": "2024-05-01"},
    {"station": {"id": "PL002", "city": "Kraków"},
     "pm25": 45.7, "date": "2024-05-01"},
]

df = pd.json_normalize(dane)
# Kolumny: station.id, station.city, pm25, date
```

Zadania do zrobienia

- Ćwiczenie 1: kodowanie znaków
- Ćwiczenie 2: Brudny CSV
- Ćwiczenie 3: Pobieranie danych z API NBP

Formaty binarne — po co?

Pliki tekstowe (CSV, JSON) są wygodne, ale mają poważne wady przy dużych zbiorach danych:

- Duży rozmiar (brak kompresji)
- Wolne wczytywanie (parsowanie tekstu znak po znaku)
- Brak typów (wszystko trzeba konwertować)
- Brak schematu

Formaty binarne rozwiązują te problemy: zapisują dane bezpośrednio w postaci, którą rozumie komputer, razem z typami i (często) ze schematem.

Cena: nie da się ich otworzyć w edytorze tekstu.

Wierszowe vs kolumnowe

Klasyczny CSV i większość baz relacyjnych zapisuje dane **wierszami**:

```
1, Anna,   Łódź,      34
2, Bartek, Warszawa, 28
3, Celina, Kraków,   45
```

Formaty **kolumnowe** (jak Parquet) zapisują te same dane kolumnami:

```
id:      1, 2, 3
imie:    Anna, Bartek, Celina
miasto:  Łódź, Warszawa, Kraków
wiek:    34, 28, 45
```

W analizach najczęściej operujemy na **całych kolumnach**.

Format kolumnowy pozwala wczytać **tylko te kolumny, których naprawdę potrzebujemy** — i lepiej je skompresować, bo wartości w kolumnie są tego samego typu.

Parquet — standard analityki

Apache Parquet to kolumnowy format binarny, dziś standard w data engineeringu i analityce.

- **Kompresja** — zwykle 5–10x mniejszy niż CSV
- **Kolumnowy** — czyta tylko potrzebne kolumny
- **Zachowuje typy** — liczby, daty, kategorie
- **Schemat** — wbudowany w plik
- **Szybkość** — 5–20x szybsze wczytywanie niż CSV


```
# Zapis i odczyt
df.to_parquet("dane.parquet", compression="snappy")
df = pd.read_parquet("dane.parquet")

# Czytanie tylko wybranych kolumn
df = pd.read_parquet("dane.parquet",
                    columns=["imie", "wiek"])
```

HDF5 — format dla danych naukowych

Hierarchical Data Format v5 — format dla dużych, hierarchicznych zbiorów naukowych. Używany m.in. w fizyce, astronomii, klimatologii, bioinformatyce.

- Wiele tabel w jednym pliku, ułożonych w drzewo
- Możliwość wczytywania fragmentów (bez ładowania całości do RAM)
- Obsługa ogromnych zbiorów (terabajty)
- Wsparcie dla metadanych (atrybuty)

```
# Zapis wielu tabel
df.to_hdf("dane.h5", key="sprzedaz", mode="w")
df2.to_hdf("dane.h5", key="klienci", mode="a")

# Odczyt z filtrowaniem (bez wczytywania całości)
df = pd.read_hdf("dane.h5", key="sprzedaz",
                 where="cena > 100")
```

Inne formaty binarne

- **Pickle** — natywny format Pythona, zapisuje dowolne obiekty (nie tylko DataFrame). Szybki, ale: niebezpieczny (wczytanie pickle z nieznanego źródła może wykonać dowolny kod), niestabilny między wersjami Pythona, nieczytelny dla innych języków. **Nie używać do długoterminowego przechowywania ani do wymiany danych.**
- **Formaty domenowe** — FITS (astronomia), NetCDF (klimat, oceanografia), GeoTIFF, LAS (LiDAR), ROOT (fizyka cząstek). Każda dziedzina nauki ma swoje formaty zoptymalizowane pod swój typ danych.

Porównanie formatów

Format	Typ	Rozmiar	Szybkość	Typy	Czytelny
CSV	tekst, wiersze	duży	wolny	nie	tak
JSON	tekst, hierarch.	duży	wolny	częściowo	tak
Parquet	binarny, kolumny	mały	bardzo szybki	tak	nie
HDF5	binarny, hierarch.	mały	szybki	tak	nie
Pickle	binarny, Python	średni	szybki	tak	nie

Reguła kciuka: CSV do wymiany z ludźmi, Parquet do pracy analitycznej, HDF5 do dużych danych naukowych, JSON do API.

Bazy danych — po co?

Pliki są wygodne, dopóki dane są małe i statyczne.

Dlaczego baza, a nie plik?

- Dane **za duże** na pamięć RAM
- **Filtrowanie po stronie serwera** — pobierasz tylko to, czego potrzebujesz
- **Wielodostęp** — wielu użytkowników jednocześnie
- **Spójność** — transakcje, ograniczenia (klucze, typy)
- **Indeksy** — szybkie wyszukiwanie w dużych zbiorach
- **Schemat** — gwarancja struktury danych

Bazy dzielimy na **relacyjne** (SQL) i **nierelacyjne** (NoSQL). W obróbce danych najczęściej spotkasz te pierwsze.

SQLite — baza w jednym pliku

SQLite to lekka baza relacyjna — cała baza to jeden plik na dysku.

Idealna do nauki, prototypów i małych aplikacji.

```
import sqlite3
import pandas as pd

conn = sqlite3.connect("baza.db")

# Wczytaj wynik zapytania jako DataFrame
df = pd.read_sql("SELECT * FROM klienci", conn)
```

```
# Bardziej złożone zapytanie
query = """
    SELECT imie, nazwisko, suma_zakupow
    FROM klienci
    WHERE miasto = 'Łódź' AND wiek > 25
    ORDER BY suma_zakupow DESC
"""
df = pd.read_sql(query, conn)

conn.close()
```


PostgreSQL

Dla większych zastosowań używamy serwerowych baz, jak PostgreSQL czy MySQL. W Pythonie standardem jest SQLAlchemy.

```
from sqlalchemy import create_engine

engine = create_engine(
    "postgresql://user:haslo@localhost:5432/mojabaza"
)
df = pd.read_sql("SELECT * FROM sprzedaz", engine)
```

Chunking

Chunking — gdy tabela nie mieści się w RAM, wczytujemy ją porcjami:

```
for chunk in pd.read_sql("SELECT * FROM duza_tabela",  
                           engine, chunksize=10000):  
    przetworzone = chunk[chunk["wartosc"] > 100]  
    przetworzone.to_parquet(f"czesc_{i}.parquet")
```

Złota zasada: filtruj po stronie bazy (w WHERE), nie po stronie Pythona. Baza robi to szybciej i nie wyśle niepotrzebnych danych przez sieć.

DuckDB — SQL dla plików

DuckDB to nowoczesna baza
analityczna — „SQLite dla analityki”.
Jeden plik, bez serwera, ale
zoptymalizowana pod zapytania
analityczne zamiast transakcyjnych.

Kluczowa cecha: potrafi czytać pliki
CSV, Parquet i JSON **bezpośrednio**,
bez importowania do bazy. Pisziesz
SQL-a na pliku, jakby był tabelą.

```
import duckdb

df = duckdb.query("""
    SELECT miasto, AVG(wiek) AS sredni_wiek
    FROM 'klienci.parquet'
    WHERE rok = 2024
    GROUP BY miasto
    ORDER BY sredni_wiek DESC
    """).df()

# Działa też na CSV
df = duckdb.query("""
    SELECT * FROM 'dane.csv'
    WHERE pm25 > 50
    """).df()
```

Zadania do zrobienia

- Ćwiczenie 4: Benchmark formatów
- Ćwiczenie 5: SQLite i pandas

Backup

Engine w read_csv

- Silnik parsujący i konwertujący: „c”, „python” lub „pyarrow”. Domyślnie jest stosowany „c”, ale nowszy „pyarrow” parsuje niektóre rodzaje plików znacznie szybciej. Silnik „python”

jest wolniejszy, ale posiada kilka funkcjonalności niedostępnych w pozostałych silnikach.

Silniki parsujące w `pandas.read_csv()`

Funkcja `read_csv` w `pandas` obsługuje trzy silniki parsujące, wybierane parametrem `engine`: `"c"`, `"python"` i `"pyarrow"`. Różnią się szybkością, zużyciem pamięci i zakresem obsługiwanych opcji.

Silnik `"c"` (domyślny)

Napisany w C, stanowi kompromis między wydajnością a funkcjonalnością. Jest szybki i obsługuje większość typowych przypadków użycia.

- Szybki i stabilny
- Niskie zużycie pamięci
- Nie obsługuje skipfooter ani separatorów opisanych wyrażeniem regularnym

Silnik "python"

Implementacja w czystym Pythonie. Najwolniejszy, ale najbardziej elastyczny.

- Obsługuje wyrażenia regularne jako separator, np. `sep=r"\s+| ; | , "`
- Obsługuje skipfooter (pomijanie wierszy od końca pliku)
- Lepsze komunikaty błędów przy nietypowych plikach
- Często 5–10 razy wolniejszy niż silnik C

Silnik "pyarrow"

Najnowszy silnik (od pandas 1.4), oparty na bibliotece Apache Arrow. Wymaga doinstalowania pakietu pyarrow.

- Bardzo szybki, często 5–20 razy szybciej niż silnik C
- Wielowątkowy — wykorzystuje wszystkie rdzenie CPU

- Świetny dla dużych plików (setki MB, GB)
- Nie obsługuje wszystkich opcji read_csv (m.in. chunksize, iterator, skipfooter)

Porównanie

Cecha	c	python	pyarrow
Szybkość	szybki	wolny	bardzo szybki
Wielowątkowość	nie	nie	tak
Regex jako separator	nie	tak	nie
skipfooter	nie	tak	nie
chunksize / iterator	tak	tak	nie
Zużycie pamięci	niskie	wysokie	niskie
Dodatkowa zależność	nie	nie	tak

Przykłady użycia

```
import pandas as pd
```

```
# Domyślnie --- silnik C
```

```
df = pd.read_csv("dane.csv")
```

```
# Szybkie wczytywanie dużego pliku
```

```
df = pd.read_csv("duzy_plik.csv", engine="pyarrow")
```

```
# Separator opisany wyrażeniem regularnym
```

```
df = pd.read_csv("dane.txt", sep=r"\s+", engine="python")
```

```
# Pomińcie ostatnich 3 wierszy (np. stopka raportu)
```

```
df = pd.read_csv("raport.csv", skipfooter=3, engine="python")
```